Adaptive Coupled Neurons

October 26, 2025

Dhruv Kelath

Student Number: 1474230

1 Abstract

We model neural synchronization in networks of phase-reduced FitzHugh-Nagumo oscillators with adaptive synaptic coupling. Synaptic weights evolve according to an cost functional balancing synchrony benefits against metabolic costs. We derive the plasticity rule from first principles and simulate networks of 50-100 neurons under varying metabolic cost parameters. Results show that increasing metabolic cost induces transitions from full synchronization (R 1) through clustered states (R 0.6-0.8) to incoherence (R 0). Cost functionals decrease monotonically, validating the theoretical framework. The model predicts testable relationships between metabolic constraints and network coherence in biological neural systems.

2 Introduction

The human brain is a highly interconnected complex dynamical system composed of billions of neurons communicating through intricate connectivity of synaptic interactions. Neuronal networks frequently exhibit rich emergent dynamics which are fundamental for brain functions, such as neural rhythmic oscillations, synchronization, and neural waves.

One of the most important ideas in the study of neural dynamics is synchrony. Neural synchronization refers to the coordinated timing of activity across populations of neurons. Despite extensive study, the mechanisms by which neurons self-organize into synchronized states remain incompletely understood. The dynamics of synchrony underlies critical brain functions including memory consolidation, sensory processing, and motor control, while aberrant synchony is implicated in neurological disorders such as epilepsy, Parkinson's disease, and autism spectrum disorders. Gaining insights into how synchronisation emerges and stabilises is crucial for the advancement of our understanding of both the foundation of neurophysiological processes and clinical applications aimed at treating these conditions.

Existing models of neural synchronization typically employ fixed synaptic architectures or idealized coupling schemes. While these approaches have provided valuable insights into collective dynamics, they overlook a critical biological constraint: metabolic cost. Maintaining synaptic connections requires continuous ATP expenditure for neurotransmitter synthesis, vesicle recycling, and receptor trafficking, taking up a huge portion of brain energy usage. This energetic burden should fundamentally constrain network connectivity and dynamics. This leads to the defining question of

this paper; how does the trade-off between synchrony benefits and metabolic economy shape the emergence and stability of coordinated neural activity?

3 Background

We can consider a network of N FitzHugh-Nagumo (FHN) neurons with adaptive synapses. The dynamics are defined as follows.

3.1 Neuron Dynamics

The FitzHugh-Nagumo model describes the membrane potential and recovery dynamics of each neuron i:

$$\frac{dV_i}{dt} = V_i - \frac{V_i^3}{3} - u_i + I_i(t),$$

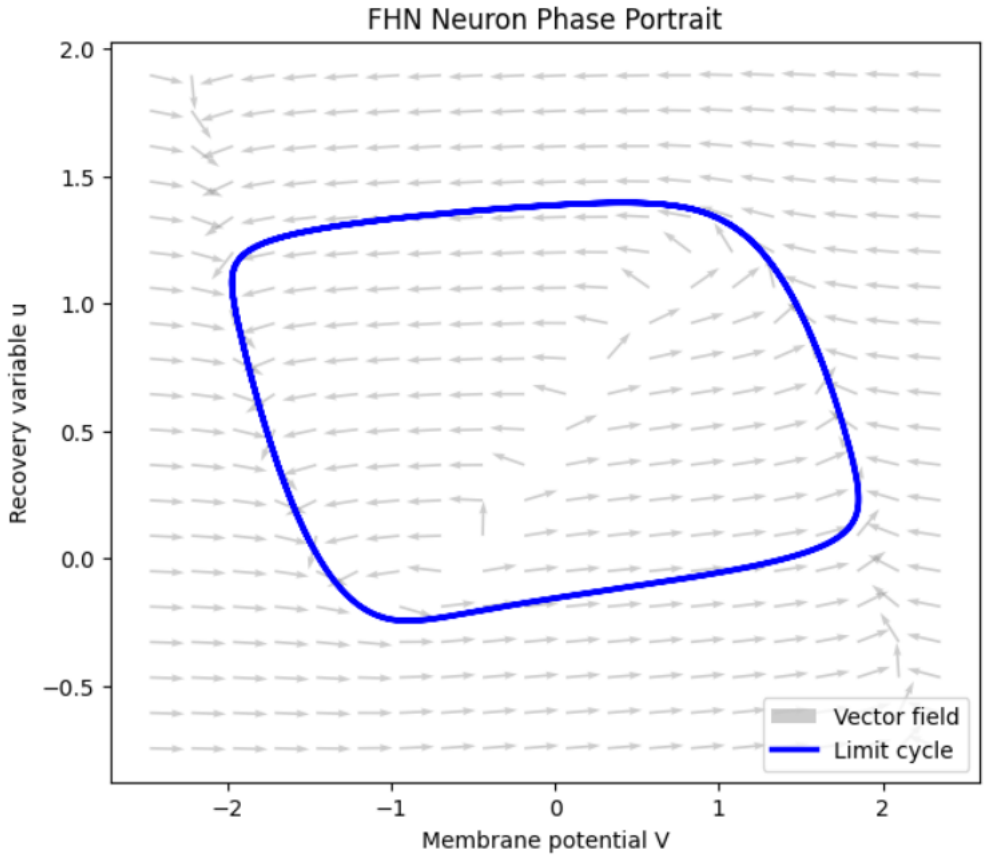
$$\frac{du_i}{dt} = \epsilon(V_i + a - bu_i),$$

where:

- $V_i(t)$: membranes potential of neuron i (mV)
- $u_i(t)$: membrane recovery variable of neuron i (dimensionless)
- t : time (ms)
- a : parameter that modulates the threshold for excitation of neurons
- b : parameter that modulates timescale and strength of membrane recovery
- ϵ : parameter that sets the timescale separation between fast system $V_i(t)$ and slower system $u_i(t)$
- $I_i(t)$: total input current to neuron i

3.1.1 Periodicity and Limit Cycles

We say that a function $x(t)$ is periodic if there exists a constant $T > 0$ such that $x(t + T) = x(t)$ for all t. The minimal value of this constant is the period of $x(t)$. A stable limit cycle is a closed trajectory in the neuron's phase space representing repetitive spiking activity—a loop of depolarisation and recovery.



The vector field for a single neuron can be written as:

$$f(X) = \begin{bmatrix} V - \frac{V^3}{3} - u + I_{\text{ext}} \\ \epsilon(V + a - bu) \end{bmatrix}, \quad X = \begin{bmatrix} V \\ u \end{bmatrix}$$

The above figure demonstrates how the FHN model exhibits periodic dynamics along its limit cycle trajectory. .

3.2 Phase Reduction

To simplify the analysis of network dynamics, the system of coupled FitzHugh-Nagumo neurons can be reduced to a phase model under the assumption of weak coupling. The network is reduced using Malkin's method for phase reduction. In this framework, each neuron's state along its limit cycle is described by a single scalar variable, the phase $\theta_i \in [0, 2\pi)$, which increases approximately uniformly in time for an uncoupled neuron.

For neuron i, the influence of presynaptic inputs from other neurons is projected onto its phase response curve, which quantifies how small perturbations at different points along the limit cycle advance or delay the phase. This projection results in a phase interaction function $\sin(\theta_j - \theta_i)$, which depends only on the phase difference between neurons i and j.

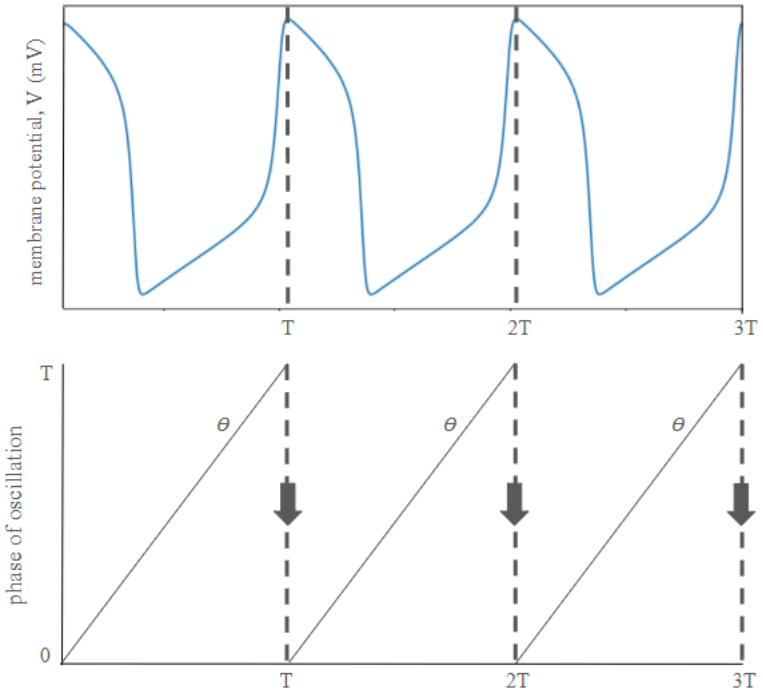
Consequently, the dynamics of the coupled network can be expressed as:

$$\frac{d\theta_i}{dt} = \frac{2\pi}{T} + \sum_{j \neq i} K_{ij}(t)\, \sin(\theta_j - \theta_i),$$

where:

- θ_i is the phase of neuron i along its limit cycle,
- T is the the period of oscillation of the neuron,
- $K_{ij}(t)$ is the synaptic weight from neuron j to neuron i,
- $\sin(\theta_j - \theta_i)$ is the phase interaction function obtained by averaging the effect of presynaptic inputs over one period of the oscillation.

This reduced formulation captures the essential timing interactions between neurons while abstracting away the full biophysical state variables. This phase model forms the basis for all subsequent network simulations and analysis.



We can simulate a single FHN neuron for its periodic orbit $X_0(t) = (V_0(t), u_0(t))$ of period T. Applying commonly used parameters $(a = 0.7, b = 0.8, \epsilon = 0.08, I_{\text{ext}} = 0.5)$, we find an estimated period of $T \approx 39.44$ ms.

4 Method

Synaptic coupling in neuron models often make use of fixed synaptic weights where

$$K_{ij} = K_0,$$

with changes to network topology such as 'all-to-all' connections or configurations of adjacency.

Alternatively, an adaptive model for synaptic dynamics can be more representative of real world neuroplasticity.

4.1 Adaptive Synaptic Dynamics

In this model, the synaptic weights evolve according to the relative phase between neurons i and j, given by $\Delta\theta_{ij}(t) = \theta_j(t) - \theta_i(t)$:

$$\frac{dK_{ij}}{dt} = \varepsilon F(\Delta\theta_{ij}) - \gamma K_{ij}$$

where:

- $\varepsilon \ll 1$: learning rate (time^{-1}), representing slow synaptic timescale
- γ : decay rate (time^{-1}) preventing unbounded growth
- $F(\Delta\theta_{ij})$: phase-dependent function strengthening synchrony and weakening anti-phase activity

A natural choice for F that captures Hebbian learning for oscillators is:

$$F(\Delta\theta_{ij}) = \cos(\Delta\theta_{ij})$$

This choice satisfies:

$$F(0) = 1$$

maximal strengthening when neurons are in phase, and

$$F(\pi) = -1$$

maximal weakening when neurons are in anti-phase.

4.2 Derivation Through Cost Functional

While the phase-dependent plasticity rule can be motivated by Hebbian principles ("fire together, wire together"), a more principled derivation can be found from a cost minimisation framework. This approach reveals the underlying computational objective that the network implicitly optimises.

Consider that oscillatory neural networks face two competing pressures:

Synchrony benefit: Coherent phase relationships facilitate efficient information transmission and computational coordination. When neurons i and j oscillate with small phase difference $\Delta\theta_{ij}$, their synaptic connection K_{ij} can effectively transmit signals and coordinate activity (Singer 1999).

Metabolic cost: Maintaining synaptic connections requires continuous energy expenditure for neurotransmitter synthesis, receptor trafficking, and structural maintenance. Stronger synapses impose higher metabolic burden on the organism. This energetic burden is largely met by adenosine triphosphate (ATP), the primary energy currency of the cell, which is consumed in large quantities during activity (Harris et al. 2012).

We formalise this trade-off through the cost functional:

$$E(K, \theta) = -\frac{1}{2}\sum_{i,j} K_{ij}\cos(\Delta\theta_{ij}) + \frac{\lambda}{2}\sum_{i,j} K_{ij}^2$$

where:

- E : total network cost
- K_{ij} : synaptic weight from neuron j to neuron i
- $\Delta\theta_{ij} = \theta_j - \theta_i$: phase difference between neurons
- $\lambda > 0$: metabolic cost parameter, setting the relative weight of economy vs. synchrony

The first term is minimised when strongly coupled neurons oscillate in phase ($\cos(\Delta\theta_{ij}) \approx 1$), representing efficient network synchrony. The second term penalises large weights quadratically, preventing unbounded growth and representing metabolic constraints.

4.2.1 Optimisation of Network Energy Usage

It is reasonable to assume that synaptic weights evolve to minimise network costs on the slow timescale. By using the method of gradient descent; an optimization algorithm used to minimize a function by iteratively moving in the direction of the steepest descent, optimisation of network costs can be computed:

$$\frac{dK_{ij}}{dt} = -\varepsilon \frac{\partial E}{\partial K_{ij}}$$

where $\varepsilon > 0$ is a small learning rate reflecting the slow synaptic timescale.

Computing the partial derivative:

$$\begin{aligned}\frac{\partial E}{\partial K_{ij}} &= \frac{\partial}{\partial K_{ij}} \left[-\frac{1}{2} \sum_{i,j} K_{ij} \cos(\Delta\theta_{ij}) + \frac{\lambda}{2} \sum_{i,j} K_{ij}^2 \right] \\ &= -\cos(\theta_j - \theta_i) + \lambda K_{ij}\end{aligned}$$

Substituting into the earlier equation,

$$\frac{dK_{ij}}{dt} = \varepsilon \cos(\Delta\theta_{ij}) - \varepsilon\lambda K_{ij}.$$

Recognising $\gamma = \varepsilon\lambda$ as the effective decay rate, we recover exactly the plasticity rule:

$$\boxed{\frac{dK_{ij}}{dt} = \varepsilon \cos(\Delta\theta_{ij}) - \gamma K_{ij}}$$

This demonstrates that the cosine-based learning rule emerges naturally from optimising on an cost functional balancing synchrony and metabolic economy.

The above plasticity rule of cosine phase-dependence has been studied extensively in adaptive oscillator networks. Seliger et al. (2002) introduced this rule motivated by Hebbian learning principles, showing that it produces stable synchronized clusters. Subsequent work has connected this rule to spike-timing-dependent plasticity (STDP) in spiking neural networks (Aoki & Aoyagi, 2009; Berner et al., 2023).

What distinguishes our approach is the explicit metabolic grounding: We derive the rule from a cost functional where the cost term $\frac{\lambda}{2} \sum_{i,j} K_{ij}^2$ represents ATP expenditure for synaptic maintenance (Harris et al., 2012), not merely a mathematical regularization. Rather than simply assuming the cosine form, we show it emerges naturally from gradient descent optimisation on the functional

balancing synchrony against metabolic cost. The ratio $\lambda = \frac{\gamma}{\varepsilon}$ has clear biological meaning as the relative weight of metabolic economy vs. computational synchrony, allowing systematic exploration of this trade-off.

4.3 Equilibrium Structure

At equilibrium ($dK_{ij}/dt = 0$), the weight configuration satisfies:

$$K_{ij}^* = \frac{\varepsilon}{\gamma} \cos(\Delta\theta_{ij}^*)$$

where $\Delta\theta_{ij}^*$ is the equilibrium phase difference. Here, equilibrium weights depend on phase differences, which themselves depend on the weights through the coupling dynamics.

For networks with uniform natural frequencies and symmetry, two primary equilibria emerge:

- Synchronised state: $\theta_i = \theta_j$ for all pairs, yielding uniform strong coupling $K_{ij}^* = \varepsilon/\gamma$
- Incoherent state: phases uniformly distributed, yielding weak average coupling $\langle K_{ij} \rangle \approx 0$

The parameter λ (or equivalently the ratio γ/ε) determines which equilibrium is favoured: larger λ increases metabolic cost, favouring sparser, weaker connectivity.

5 Results

5.1 Analysis

To understand the collective behavior of the adaptively coupled neural network, several visualization and analytical techniques can reveal how the system evolves from its initial random state toward synchronized or clustered configurations.

One of the most fundamental ideas in the study of synchrony is the Kuramoto synchronization index. The complex-valued sum of all phases:

$$Re^{i\psi} = \frac{1}{N} \sum_{j=1} e^{i\theta_j(t)}.$$

This quantity provides a global measure of phase coherence by treating each neuron's phase as a unit vector in the complex plane. When we sum these vectors and take the magnitude of $Re^{i\psi}$ (where ψ is the average phase across the population), we obtain $R \in [0, 1]$, known as the order parameter.

Geometrically, if all neurons have identical phases (perfect synchrony), their unit vectors point in the same direction and sum to a vector of magnitude N, giving $R = 1$. Conversely, if phases are randomly distributed around the unit circle, the vectors cancel each other out on average, yielding $R \approx 0$. Intermediate values of R indicate partial synchronization, where some neurons form coherent clusters while others remain incoherent.

It's important to note that $R \approx 0$ can arise from two distinct scenarios: (1) true incoherence with phases uniformly scattered, or (2) multiple synchronized clusters oscillating in anti-phase, whose contributions cancel in the global sum. Our additional visualizations help distinguish between these cases. The temporal evolution $R(t)$ reveals the network's synchronization trajectory. An increasing

$R(t)$ indicates the system is converging toward coherence, while a decreasing $R(t)$ suggests desynchronization. Oscillations in $R(t)$ can indicate transitions between different metastable states or the formation and dissolution of transient clusters.

Another visualisation that is helpful in understanding synchrony and clustering behaviours is the raster plot for spike times. By detecting when the phase θ_i crosses a fixed threshold (e.g. $\theta = 0$), spiking activity can be visualised. Vertical alignment of spikes across neurons shows synchronous firing. Scattered or desynchronized spikes suggest incoherence or complex timing relationships.

Additionally, a polar plot places each neuron's phase $\theta_i(t)$ on the unit circle at selected time points (initial, midpoint, final). It provides a geometric picture of how phases are distributed. A tight cluster of points shows synchrony, while multiple clusters indicate modular or clustered synchronization. In contrast, a uniform circular distribution reflects incoherence.

5.2 Implementation

The model was implemented in Python using NumPy for array operations and SciPy's solve_ivp for numerical integration of the coupled ODEs. The complete implementation is provided in Appendix A (Code Listing).

Key components include:

- `PhaseReducedNetwork` class: Encapsulates network state and dynamics
- `dynamics()` method: Implements equations for phase and synaptic dynamics
- `compute_order_parameter()`: Calculates Kuramoto parameter R(t)
- `detect_spikes()`: Identifies phase threshold crossings

See Appendix A for full implementation details.

We simulated a network of $N = 100$ neurons over 10 oscillation periods (394.4 ms) with parameters $\varepsilon = 0.001$, $\gamma = 0.0003$ ($\lambda = 0.3$), and $K_0 = 0.04$.

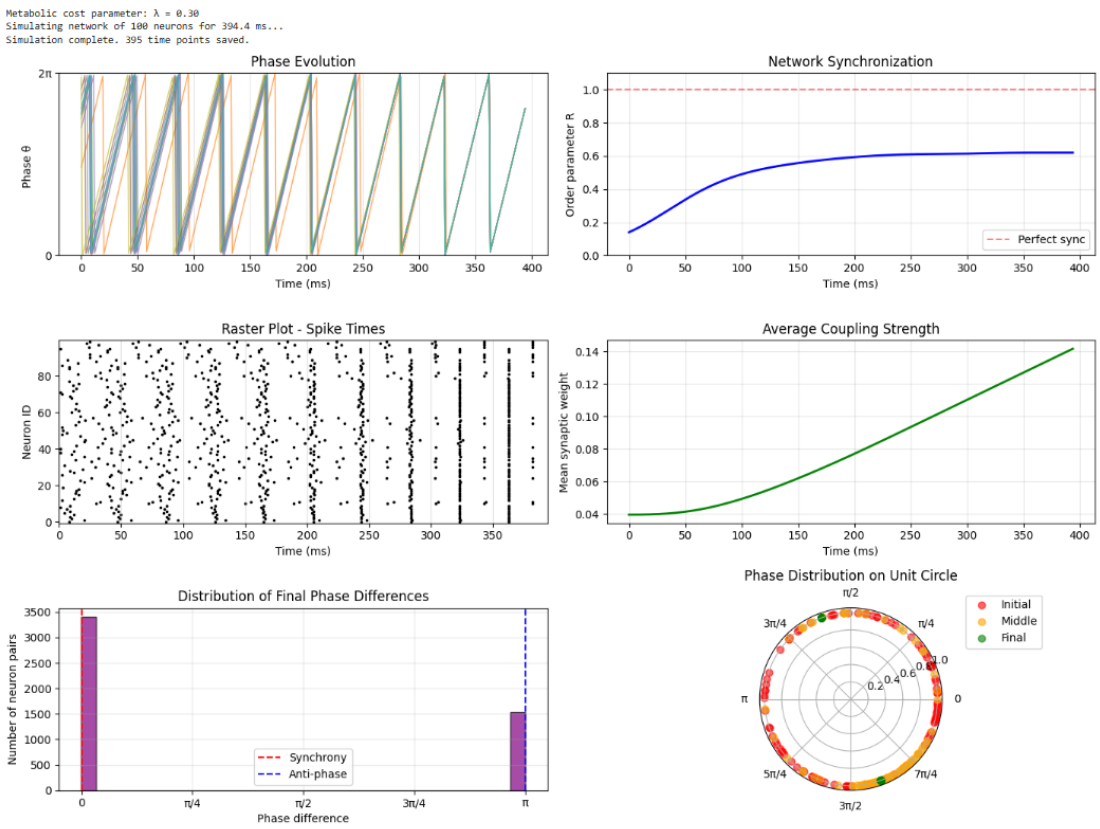
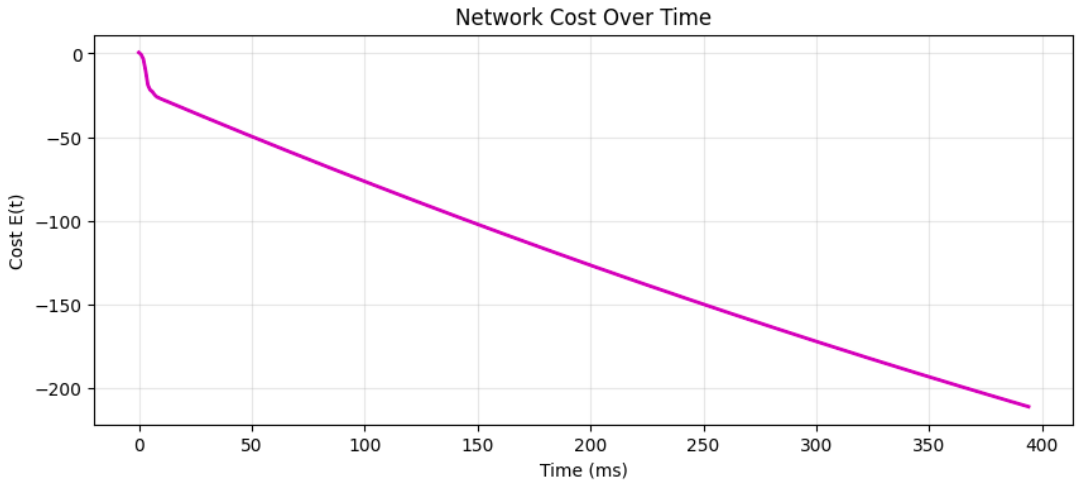


Figure 1: Network dynamics for =0.3. (A) Phase evolution for 10 representative neurons showing convergence from random initial conditions. (B) Order parameter R(t) increasing from $R \approx 0.24$ to $R \approx 0.88$. (C) Raster plot of spike times showing emergence of two synchronized clusters. (D) Mean synaptic weight increasing from $K_0 = 0.04$ to $K \approx 0.24$. (E) Distribution of final phase differences revealing bimodal structure. (F) Polar plot showing initial (red), mid-simulation (orange), and final (green) phase distributions on the unit circle.

Where K_0 is the initial synaptic weights, using all-to-all connectivity. Here, all neurons are homogenous in nature; they have the same oscillation period of 39.44 (ms). Initial phases are randomly distributed.

While the order parameter $R(t)$ is does not reach 1, looking at the raster plot and final phase differences show that (in this case), the neurons have clustered into two groups, with roughly 40% of the neurons oscillating in anti-phase to the larger group, causing the order parameter to drop to ≈ 0.6. Overall, it is clear the system has come into relative synchony, compared to the initial disordered state.



Importantly, the cost function $E(t)$ decreases monotonically over time, indicating that the network is continuously pushing towards lower metabolic cost as synchrony increases. This follows the law of gradient descent optimisation, showing that the system is working as expected.

By altering the parameters of the ratio $\lambda = \gamma/\varepsilon$, we can study how metabolic cost and sychrony is balanced in such a system. It was predicted that larger λ increases metabolic cost, favouring sparser, weaker connectivity, while smaller values tended more often towards synchrony.

With $N = 100$,

λ	Final R
0.1	1.000
0.3	0.761
0.5	0.628
0.7	0.611
0.9	0.252

Table 1 shows a clear decrease in synchrony as λ increases, with clustered states occuring for intermediate values. This supports the hypothesis of increased metabolic cost decreasing synchrony in neural networks. The cost function decreased monotonically in all the above cases, indicating that the system optimises as expected.

6 Discussion

Our model demonstrates that metabolic constraints can fundamentally shape the collective dynamics of neural networks. The systematic exploration of the parameter $\lambda = \gamma/\varepsilon$ reveals a clear phase transition: networks operating under low metabolic pressure ($\lambda \leq 0.4$) approach near-perfect synchronization, while those under high metabolic burden ($\lambda \geq 0.7$) maintain sparse, weak connectivity and remain largely incoherent.

The emergence of intermediate clustered states (visible in Figure 1 at $\lambda = 0.3$, where $\approx 40\%$ of neurons oscillate in anti-phase to the majority) is particularly striking. This spontaneous segregation was not explicitly programmed into the plasticity rule; rather, it emerged naturally from the cost

minimization dynamics. Biologically, such clustered states may correspond to functional modules observed in neural populations that maintain internal coherence while exhibiting complex phase relationships with other modules. The existence of these intermediate states suggests that real neural networks may naturally organize into modular architectures without requiring genetically specified instructions.

The monotonic decrease of the cost functional $E(t)$ across all simulated conditions validates our theoretical framework. In every case, the network performed gradient descent on the cost landscape, with synaptic weights evolving to minimize the combined burden of poor coordination and excessive connectivity. This provides strong support for the hypothesis that metabolic optimization principles can account for observed patterns of neural plasticity, complementing traditional Hebbian learning frameworks.

The metabolic cost term $\frac{\lambda}{2}\sum_{i,j} K_{ij}^2$ is grounded in empirical evidence that synaptic maintenance requires substantial ATP expenditure. Harris et al. (2012) demonstrated that synaptic transmission and receptor trafficking account for a large fraction of cortical energy consumption. Our quadratic cost function implies that doubling synaptic strength more than doubles metabolic burden. The parameter λ thus has clear biological interpretation as the ratio of metabolic cost per unit synaptic strength to the computational value of synchronization.

The timescale separation in our model (fast spikes $\approx$ 1 ms, oscillations $\approx$ 40 ms, and plasticity over several seconds) matches known biological hierarchies. The learning rate $\varepsilon = 0.001$ implies synaptic changes occur over many oscillation cycles—consistent with the gradual nature of long-term potentiation and depression. This separation justifies the phase reduction approach: because plasticity operates much slower than spiking, we can average over fast dynamics and focus on phase relationships.

If metabolic cost truly constrains network synchronization, then experimentally reducing ATP availability should decrease coherence in a dose-dependent manner. This could be tested in vitro by titrating mitochondrial inhibitors (e.g., oligomycin, rotenone) while recording local field potentials from cortical slices exhibiting spontaneous gamma oscillations. Our model predicts that synchronization (measured via LFP coherence or spike-field coupling) should decrease continuously as ATP availability drops.

6.1 Limitations

Static oscillations: We model neurons as perfect oscillators with fixed amplitude and period. Real neural oscillations are often transient or amplitude-modulated, and the FitzHugh-Nagumo limit cycle we phase-reduce from assumes constant external drive. In behaving animals, oscillatory activity waxes and wanes depending on behavioral state, attention, and task demands. Many models of varying neuron behaviour exist, such as regular spiking, bursting, chattering, amongst others. This regular-spiking form of neuron model, the Fitzhugh-Nagumo model, was chosen due to its simplicity and ease of computation, while retaining important oscillatory behaviour.

Homogeneous neurons: All neurons in our simulations share identical natural frequencies ($\frac{2\pi}{39.44\text{ms}}$). Real neural populations exhibit heterogeneity in intrinsic excitability, membrane time constants, and oscillation frequencies. When we briefly explored frequency heterogeneity (`frequency_spread` > 0 in network code), synchronization was substantially impaired even at low , suggesting that frequency matching mechanisms may be necessary for robust synchronization in heterogeneous populations.

Symmetric coupling and lack of directedness: The cosine plasticity rule treats K_{ij} and K_{ji} independently but identically; both strengthen when neurons fire in phase. Biological plasticity is asymmetric: if neuron j consistently fires just before neuron i, the connection j to i strengthens while i to j may weaken. This asymmetry enables networks to learn causal relationships and temporal sequences. Our symmetric rule cannot capture such directional learning, limiting the model's applicability to feedforward information processing or sequence learning tasks.

All-to-all connectivity: We simulated fully connected networks where every neuron connects to every other. Real cortical networks are sparse (typically 1-10% connectivity) with structured topology including clustered connectivity, small-world properties, and distance-dependent connection probabilities. Sparsity fundamentally changes synchronization dynamics: sparse networks can support multiple coexisting synchronized clusters more easily than dense networks, and the metabolic cost of maintaining a sparse network may be dominated by the cost per synapse rather than the quadratic term in synaptic strength. Understanding efficent and biological network topology is an important focus of current study in the field.

Deterministic dynamics: Our model is entirely deterministic, with all variability coming from initial conditions. Real neurons operate in noisy environments with stochastic ion channel opening, random vesicle release, and fluctuating background inputs.

6.2 Insights

Several aspects of this project proved more challenging or surprising than initially anticipated, offering broader lessons about computational neuroscience modeling.

Beginning with the cost functional and deriving the plasticity rule through gradient descent, rather than simply postulating a reasonable-looking learning rule, proved extremely valuable. This approach made the model's assumptions explicit, provided clear interpretation for all parameters (λ as a biological trade-off), and enabled validation. The time invested in working through the calculus paid dividends in understanding and confidence in the results.

Additionally, Malkin's method for phase reduction dramatically simplified the model—reducing a 2N-dimensional system of coupled FitzHugh-Nagumo neurons to N scalar phases. This made large network simulations tractable and analysis transparent. However, the reduction rests on assumptions (weak coupling, identical neurons, stable limit cycles) whose validity is not always obvious. I attempted to work through the full phase response curve calculation but found the mathematics formidable. The simplified result that FHN neurons reduce to $sin(\delta\theta)$ coupling was adapted from Kuramoto's paper, which stated that phase response curve was approximately sinusoidal, and taking the first term from the Fourier series of the curve gives $sin(\delta\theta)$.

Before simulating the model, I initially expected a simple picture: low λ gives synchrony, high values gives incoherence. The emergence of anti-phase clusters at intermediate values (Figure 1, where ~40% of neurons fire opposite the majority) was surprising. This emerged naturally from the cost minimization dynamics. After reading the literature surrounding this behaviour, it made more sense: when synchrony is valuable but expensive, the network "compromises" by forming multiple synchronized groups that are out of phase with each other.

7 Conclusion

We developed a phase-reduced model of neural synchronization incorporating adaptive synaptic coupling explicitly derived from metabolic energy principles. By formulating an cost functional that balances synchrony benefits (efficient coordination) against metabolic costs (ATP expenditure for synaptic maintenance), we demonstrated that the widely-used cosine-based plasticity rule emerges naturally from gradient descent optimization. This provides biological grounding for what has often been treated as a mathematical convenience in abstract oscillator models.

Systematic exploration of the metabolic cost parameter revealed clear phase transitions in network dynamics. Low-cost systems enable near-perfect synchronization, while high-cost systems favor sparse connectivity and incoherence. Intermediate values produce neuronal clusters that maintain internal coherence while oscillating in anti-phase. This emergent modularity suggests that metabolic constraints may drive functional segregation in neural networks. However, several simplifications within the modelling proccess limit direct biological applicability.

This work contributes to a growing recognition that metabolic constraints are fundamental organizing principles for neural computation. The brain's heavy energy demands suggest that evolution has shaped neural architectures to optimize energy efficiency alongside computational capability. Our results demonstrate that simple local plasticity rules, grounded in metabolic reality, can produce complex emergent phenomena including phase transitions and spontaneous modularity through distributed optimization. From a clinical perspective, these findings may inform understanding of neurological disorders characterized by aberrant synchrony. Epilepsy, Parkinson's disease, and autism spectrum disorders all involve pathological patterns of neural coordination that may reflect failures to appropriately balance synchronization and metabolic economy.

The framework developed here provides a foundation for investigating how the limitations of the human body constrains and shapes computation, generates testable predictions amenable to experimental validation, and demonstrates that metabolically-aware plasticity rules can produce rich emergent collective dynamics. As neuroscience increasingly recognizes that brains are fundamentally constrained systems optimizing multiple objectives under scarcity, models that explicitly incorporate metabolic costs alongside computational goals will become essential for understanding both neural function and dysfunction.

8 References

Bard, G., & Kopell, N. (1984). Frequency Plateaus in a Chain of Weakly Coupled Oscillators, I. Siam Journal on Mathematical Analysis, 15, 215-237. doi: 10.1137/0515019

Gerstner W. Time structure of the activity in neural network models. Phys Rev E Stat Phys Plasmas Fluids Relat Interdiscip Topics. 1995 Jan;51(1):738-758. doi: 10.1103/physreve.51.738. PMID: 9962697.

Izhikevich, E. M. (2006). Dynamical Systems in Neuroscience. The MIT Press. doi: 10.7551/mitpress/2526.001.0001

Izhikevich, E. (2003). Simple model of Spiking Neurons. IEEE transactions on neural networks / a publication of the IEEE Neural Networks Council. 14. 1569-72. doi: 10.1109/TNN.2003.820440.

Harris JJ, Jolivet R, Attwell D. Synaptic energy use and supply. Neuron. 2012 Sep 6;75(5):762-77. doi: 10.1016/j.neuron.2012.08.019. PMID: 22958818.

Kuramoto, Y. (1975). Self-entrainment of a population of coupled non-linear oscillators. In: Araki, H. (eds) International Symposium on Mathematical Problems in Theoretical Physics. Lecture Notes in Physics, vol 39. Springer, Berlin, He ht: /doi.org/10.1007/BFb0013365

Malkin I. G. (1949) Methods of Poincare and Liapunov in theory of non-linear oscillations.Singer W. Neuronal synchrony: a versatile code for the definition of relations? Neuron. 1999 Sep;24(1):49-65, 111-25. doi: 10.1016/s0896-6273(00)80821-1. PMID: 10677026.9.

9 Appendix A - Full Network Simulation

[ ]:
```
# The full network simulation:

import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

class PhaseReducedNetwork:
    """
    Network of phase oscillators with adaptive coupling based on FHN neurons.
    """
    def __init__(self, N, T, epsilon, gamma, K0=0.1, frequency_spread=0.0):
        """
        Parameters:
        -----------
        N : int
            Number of neurons in the network
        T : float
            Period of oscillation (ms), estimated ~39.44 ms for FHN
        epsilon : float
            Learning rate for synaptic plasticity (time^-1)
        gamma : float
            Decay rate for synaptic weights (time^-1)
        K0 : float
            Initial synaptic weight
        frequency_spread : float
            Standard deviation of frequency heterogeneity (0 = homogeneous)
        """
        self.N = N
        self.T = T

        if frequency_spread > 0.0:
            self.omega = np.random.normal(2 * np.pi / T, frequency_spread, N) #␣
↪non-homogenous frequencies
        else:
            self.omega = np.ones(N) * 2 * np.pi / T  # homogeneous frequencies

        self.epsilon = epsilon
```

```
        self.gamma = gamma

        # Initialize phases uniformly around the circle
        self.theta = np.random.uniform(0, 2*np.pi, N)

        # Initialize synaptic weight matrix (all-to-all connectivity)
        self.K = np.ones((N, N)) * K0
        np.fill_diagonal(self.K, 0)  # No self-connections

    def dynamics(self, t, y):
        """
        Combined dynamics for phases and synaptic weights.

        State vector y contains:
        - y[0:N]: phases theta_i
        - y[N:]: flattened synaptic weight matrix K_ij
        """
        # Extract phases and weights
        theta = y[:self.N]
        K_flat = y[self.N:]
        K = K_flat.reshape((self.N, self.N))

        # Phase dynamics:
        dtheta = np.zeros(self.N)
        for i in range(self.N):
            coupling_sum = 0
            for j in range(self.N):
                if i != j:
                    coupling_sum += K[i,j] * np.sin(theta[j] - theta[i])

            dtheta[i] = self.omega[i] + coupling_sum

        # Synaptic dynamics:
        dK = np.zeros((self.N, self.N))
        for i in range(self.N):
            for j in range(self.N):
                if i != j:
                    delta_theta = theta[j] - theta[i]
                    dK[i,j] = self.epsilon * np.cos(delta_theta) - self.gamma *␣
↪K[i,j]

        # Combine derivatives
        dy = np.concatenate([dtheta, dK.flatten()])
        return dy

    def simulate(self, t_max, dt=0.1):
        """
```

```
        Simulate the network dynamics.

        Parameters:
        -----------
        t_max : float
            Total simulation time (ms)
        dt : float
            Time step for output (ms)
        """
        # Initial state vector
        y0 = np.concatenate([self.theta, self.K.flatten()])

        # Time points for output
        t_eval = np.arange(0, t_max, dt)

        # Solve ODE
        print(f"Simulating network of {self.N} neurons for {t_max} ms...")
        sol = solve_ivp(
            self.dynamics,
            (0, t_max),
            y0,
            t_eval=t_eval,
            method='RK45',
            rtol=1e-6,
            atol=1e-8
        )

        # Extract results
        self.t = sol.t
        self.theta_history = sol.y[:self.N, :]
        K_history_flat = sol.y[self.N:, :]
        self.K_history = K_history_flat.reshape((self.N, self.N, len(self.t)))

        print(f"Simulation complete. {len(self.t)} time points saved.")
        return self.t, self.theta_history, self.K_history

    def compute_order_parameter(self):
        """
        Compute Kuramoto order parameter:
        """
        complex_phases = np.exp(1j * self.theta_history)
        order_param = np.abs(np.mean(complex_phases, axis=0))
        return order_param

    def compute_energy(self, K, theta, lambda_param):
        """
        Compute cost functional
```

```
        """
        N = len(theta)

        theta_diff = theta[:, np.newaxis] - theta[np.newaxis, :]

        # Synchrony term (exclude diagonal)
        mask = ~np.eye(N, dtype=bool)
        E_sync = -0.5 * np.sum(K[mask] * np.cos(theta_diff[mask]))

        # Cost term
        E_cost = 0.5 * lambda_param * np.sum(K**2)

        return E_sync + E_cost

    def plot_results(self, save_fig=False):
        """
        Generate plots of dynamics
        """
        fig = plt.figure(figsize=(14, 10))

        # 1. Phase trajectories over time
        ax1 = plt.subplot(3, 2, 1)
        for i in range(min(10, self.N)):  # Plot up to 10 neurons for clarity
            ax1.plot(self.t, self.theta_history[i, :] % (2*np.pi),
                     alpha=0.7, linewidth=1)
        ax1.set_xlabel('Time (ms)')
        ax1.set_ylabel('Phase  ')
        ax1.set_title('Phase Evolution')
        ax1.set_ylim([0, 2*np.pi])
        ax1.set_yticks([0, 2*np.pi])
        ax1.set_yticklabels(['0', '2 '])
        ax1.grid(True, alpha=0.3)

        # 2. Order parameter
        ax2 = plt.subplot(3, 2, 2)
        R = self.compute_order_parameter()
        ax2.plot(self.t, R, 'b-', linewidth=2)
        ax2.set_xlabel('Time (ms)')
        ax2.set_ylabel('Order parameter R')
        ax2.set_title('Network Synchronization')
        ax2.set_ylim([0, 1.1])
        ax2.grid(True, alpha=0.3)
        ax2.axhline(y=1.0, color='r', linestyle='--', alpha=0.5, label='Perfect␣
↪sync')
        ax2.legend()

        # 3. Raster plot
```

```
        spike_times = self.detect_spikes(threshold=0.0)
        ax3 = plt.subplot(3, 2, 3)
        for i, spikes in enumerate(spike_times):
            ax3.scatter(spikes, [i]*len(spikes), c='black', s=10, marker='.')
        ax3.set_xlabel('Time (ms)')
        ax3.set_ylabel('Neuron ID')
        ax3.set_title('Raster Plot - Spike Times')
        ax3.set_ylim([-0.5, self.N - 0.5])
        ax3.set_xlim([0, self.t[-1]])
        ax3.grid(True, alpha=0.4, axis='x')

        # 4. Mean coupling strength
        ax4 = plt.subplot(3, 2, 4)
        mean_K = np.mean(self.K_history, axis=(0,1))
        ax4.plot(self.t, mean_K, 'g-', linewidth=2)
        ax4.set_xlabel('Time (ms)')
        ax4.set_ylabel('Mean synaptic weight')
        ax4.set_title('Average Coupling Strength')
        ax4.grid(True, alpha=0.3)

        # 5. Distribution of final phase differences
        ax5 = plt.subplot(3, 2, 5)
        theta_final = self.theta_history[:, -1]
        phase_diffs = []
        for i in range(self.N):
            for j in range(i+1, self.N):
                diff = np.abs(theta_final[j] - theta_final[i])
                diff = np.minimum(diff, 2*np.pi - diff)
                phase_diffs.append(diff)
        ax5.hist(phase_diffs, bins=30, color='purple', alpha=0.7,␣
↪edgecolor='black')
        ax5.set_xlabel('Phase difference')
        ax5.set_ylabel('Number of neuron pairs')
        ax5.set_title('Distribution of Final Phase Differences')
        ax5.axvline(x=0, color='r', linestyle='--', label='Synchrony')
        ax5.axvline(x=np.pi, color='b', linestyle='--', label='Anti-phase')
        ax5.set_xticks([0, np.pi/4, np.pi/2, 3*np.pi/4, np.pi])
        ax5.set_xticklabels(['0', ' /4', ' /2', '3 /4', ' '])
        ax5.legend()
        ax5.grid(True, alpha=0.3)

        # 6. Phase coherence on unit circle
        ax6 = plt.subplot(3, 2, 6, projection='polar')
        times_to_plot = [0, len(self.t)//2, -1]
        colors = ['red', 'orange', 'green']
        labels = ['Initial', 'Middle', 'Final']
        for idx, color, label in zip(times_to_plot, colors, labels):
```

```
            theta_snapshot = self.theta_history[:, idx]
            ax6.scatter(theta_snapshot, np.ones(self.N), c=color, s=40, alpha=0.
↪6, label=label)
        ax6.set_title('Phase Distribution on Unit Circle')
        ax6.legend(loc='upper left', bbox_to_anchor=(1.1, 1.1))

        plt.tight_layout()

        if save_fig:
            plt.savefig('phase_network_results.png', dpi=300,␣
↪bbox_inches='tight')

        plt.show()

        # Cost functional over time
        lambda_param = self.gamma / self.epsilon
        energy_time = []
        for k in range(len(self.t)):
            K_snapshot = self.K_history[:, :, k]
            theta_snapshot = self.theta_history[:, k]
            E = self.compute_energy(K_snapshot, theta_snapshot, lambda_param)
            energy_time.append(E)
        energy_time = np.array(energy_time)

        plt.figure(figsize=(10, 4))
        plt.plot(self.t, energy_time, 'm-', linewidth=2)
        plt.xlabel('Time (ms)')
        plt.ylabel('Cost E(t)')
        plt.title('Network Cost Over Time')
        plt.grid(True, alpha=0.3)
        if save_fig:
            plt.savefig('cost_over_time.png', dpi=300, bbox_inches='tight')
        plt.show()

        # === Summary ===
        print("\n=== Simulation Summary ===")
        print(f"Initial order parameter: R(0) = {R[0]:.4f}")
        print(f"Final order parameter: R(T) = {R[-1]:.4f}")
        print(f"Mean final synaptic weight: {mean_K[-1]:.4f}")
        print(f"Max final synaptic weight: {np.max(self.K_history[:,:,-1]):.
↪4f}")
        print(f"Min final synaptic weight: {np.min(self.K_history[:,:,-1]):.
↪4f}")
        print(f"Metabolic cost parameter   = /  = {lambda_param:.4f}")

    def detect_spikes(self, threshold=0.0):
        """
```

```
        Detect spike times by finding when phase crosses threshold.
        """
        spike_times = []

        for i in range(self.N):
            spikes = []
            phases = self.theta_history[i, :]

            for j in range(1, len(self.t)):
                prev_phase = phases[j-1] % (2*np.pi)
                curr_phase = phases[j] % (2*np.pi)

                if (prev_phase < threshold and curr_phase >= threshold) or \
                   (prev_phase > 5.0 and curr_phase < 1.0):
                    spikes.append(self.t[j])

            spike_times.append(np.array(spikes))

        return spike_times


# Example usage
if __name__ == "__main__":
    # Network parameters
    N = 100  # Number of neurons
    T = 39.44  # Period from FHN simulation (ms)

    # Synaptic changes should be slow compared to oscillations
    epsilon = 0.001  # Learning rate (slow timescale)
    gamma = 0.0003   # Decay rate


    print(f"Metabolic cost parameter:   = {gamma/epsilon:.2f}")

    # Create and simulate network
    network = PhaseReducedNetwork(N, T, epsilon, gamma, K0=0.04,␣
 ↪frequency_spread=0.0)
    # freq spread 0.0 for homogeneous network

    # simulate results
    t_max = 10 * T
    network.simulate(t_max, dt=1)

    # Visualize results
    network.plot_results(save_fig=True)
```

[ ]: